

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN

ĐỀ TÀI:
HIỂN THỊ CHẠY CHỮ TRÊN LED MATRIX 32X8
SỬ DỤNG ESP8266 QUA MOBILE APP

Môn học: NT131.P22 – Hệ thống nhúng Mạng không dây

Giảng viên hướng dẫn: Lê Anh Tuấn

Thực hiện bởi nhóm 11, bao gồm:

- | | | |
|-------------------------|----------|-------------|
| 1. Nguyễn Thái Bình | 21521877 | Trưởng nhóm |
| 2. Ngô Thị Mai Anh | 23520053 | Thành viên |
| 3. Trần Ngọc Thúy Vy | 23521837 | Thành viên |
| 4. Bùi Nguyễn Tường Vân | 23521768 | Thành viên |

MỤC LỤC

MỤC LỤC	2
TÓM TẮT.....	4
Chương I. TỔNG QUAN	5
1. Tổng quan	5
1.1. Định nghĩa	5
1.2. Thành phần	5
1.3. Cách hoạt động	5
2. Triển khai.....	6
2.1. Mục tiêu Triển khai	6
2.2. Kết nối phần cứng.....	6
2.3. Lập trình ESP8266	6
2.4. Thiết kế ứng dụng Flutter	6
2.5. Hoạt động hệ thống	6
Chương II. THIẾT KẾ HỆ THỐNG	7
1. Kiến trúc tổng thể hệ thống	7
2. Mô tả chi tiết luồng xử lý	7
3. Vai trò các thành phần	7
Chương III. LẬP TRÌNH PHẦN CỨNG	8
1. Cấu hình Wifi và thiết lập Web Server trên ESP8266.....	8
1.1. Kết nối WiFi	8
1.2. Thiết lập Web Server	8
2. Điều khiển LED Matrix.....	9
2.1. Sử dụng thư viện.....	9
2.2. Khởi tạo LED Matrix	9
2.3. Bộ font hiển thị	10
3. Nhận dữ liệu từ app và hiển thị lên ma trận LED	10
3.1. Mã xử lý HTTP Request.....	10
3.2. Hiển thị văn bản động (scroll text).....	11
4. Xử lý hiệu ứng	12
5. Hiệu ứng hiển thị chữ trên LED matrix.....	13

5.1. Xử lý văn bản hiển thị	13
5.2. Điều chỉnh tốc độ chạy chữ	14
5.3. Điều chỉnh độ sáng	14
5.4. Thiết lập số lần lặp chạy chữ	15
5.5. Bật/tắt nhấp nháy	15
5.6. Lệnh dừng hiển thị.....	16
Chương IV. LẬP TRÌNH ỨNG DỤNG FLUTTER	17
1. Giới thiệu	17
2. Mục tiêu ứng dụng Flutter	17
3. Giao diện người dùng (UI)	17
3.1. Cấu trúc giao diện bao gồm các thành phần:.....	17
3.2. Thiết kế giao diện trực quan và dễ thao tác:.....	18
4. Luồng xử lý dữ liệu	18
5. Giải thích mã nguồn chính	18
5.1. Các trường dữ liệu điều khiển	18
5.2. Hàm gửi dữ liệu tới ESP8266.....	19
5.3. Tạo URI từ các tham số	19
6. Cài đặt thư viện và cấu hình Flutter	19
7. Kiểm thử và triển khai	19
7.1. Yêu cầu	19
7.2. Thử nghiệm.....	19
Chương V. KẾT LUẬN	20
Kinh nghiệm và bài học rút ra	20
Hạn chế và khó khăn	20
Hướng phát triển trong tương lai.....	20

TÓM TẮT

Đồ án này trình bày quá trình thiết kế và triển khai hệ thống hiển thị chữ chạy trên LED Matrix 32x8 thông qua điều khiển từ ứng dụng Flutter. Sử dụng module ESP8266 kết nối WiFi để nhận dữ liệu từ ứng dụng gửi tới, sau đó hiển thị lên LED ma trận. Hệ thống hoạt động ổn định, giao diện đơn giản và dễ sử dụng.

Chương I. TỔNG QUAN

1. Tổng quan

1.1. Định nghĩa

a. Nội dung chính của đề tài là gì?

- Xây dựng hệ thống hiển thị chữ động (chạy chữ) trên LED Matrix 32x8.
- Điều khiển nội dung hiển thị thông qua ứng dụng di động Flutter.
- Giao tiếp giữa thiết bị và app được thực hiện qua kết nối WiFi sử dụng module ESP8266 NodeMCU.

b. Lý do chọn đề tài

- Phổ biến trong thực tế: bảng quảng cáo, hiển thị thông tin.
- Ứng dụng công nghệ IoT và mobile app kết hợp, mang tính thực tiễn và học thuật cao.
- Rèn luyện kỹ năng: lập trình nhúng (ESP8266), xử lý giao tiếp không dây, lập trình giao diện Flutter.

c. Liên quan đến môn học

- Kết hợp kiến thức các môn: Hệ thống nhúng, Mạng máy tính, Lập trình di động, Internet of Things (IoT).
- Vận dụng thực tế các kỹ thuật: điều khiển phần cứng, giao tiếp HTTP, lập trình client-server.

1.2. Thành phần

a. Phần cứng:

- **ESP8266 NodeMCU:** board vi điều khiển tích hợp WiFi, đóng vai trò Web Server nhận lệnh điều khiển từ app.
- **LED MATRIX Max7219:** ma trận LED 32x8 gồm 4 module 8x8 dùng để hiển thị văn bản.

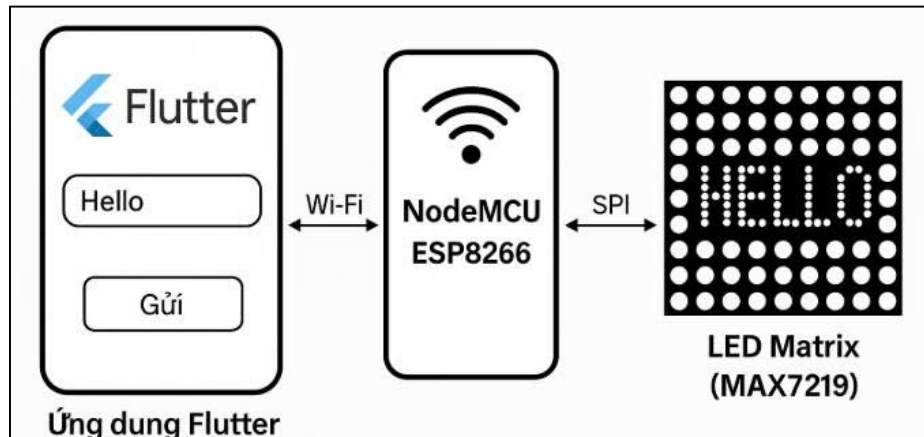
b. Phần mềm:

- **Arduino IDE:** dùng để lập trình, nạp firmware cho ESP8266.
- **Flutter SDK:** phát triển ứng dụng điều khiển trên điện thoại (Android/iOS).

1.3. Cách hoạt động

- App Flutter gửi văn bản đến ESP8266 thông qua HTTP (hoặc thông qua một Web form được host trên ESP8266).
- ESP8266 nhận dữ liệu qua endpoint HTTP và xử lý, chuyển chuỗi ký tự thành dạng hiển thị cuộn.
- Module LED Matrix nhận dữ liệu từ ESP8266 thông qua giao tiếp SPI và hiển thị chữ chạy từ phải sang trái.

- Hệ thống hoạt động như một mô hình client-server thu nhỏ, trong đó app đóng vai trò client, còn ESP8266 là server xử lý và hiển thị dữ liệu.



Hình 1. Mô hình hoạt động

2. Triển khai

2.1. Mục tiêu Triển khai

Mục tiêu của phần triển khai là xây dựng một hệ thống hoàn chỉnh có khả năng hiển thị chữ trên LED Matrix thông qua điều khiển từ một ứng dụng di động. Hệ thống sẽ bao gồm các bước từ thiết kế mạch, lập trình phần cứng, đến phát triển ứng dụng di động.

2.2. Kết nối phần cứng

Kết nối **ESP8266 NodeMCU** với **LED Matrix 32x8 (MAX7219)** qua các chân giao tiếp (DIN, CS, CLK).

2.3. Lập trình ESP8266

Dùng **Arduino IDE** để:

- Cấu hình kết nối WiFi.
- Thiết lập **Web Server** nhận chuỗi văn bản từ app.
- Xử lý và hiển thị chuỗi lên LED Matrix bằng thư viện MD_MAX72XX.

2.4. Thiết kế ứng dụng Flutter

- Giao diện gồm **ô nhập văn bản** và **nút gửi**.
- Khi người dùng bấm nút, ứng dụng gửi dữ liệu văn bản đến ESP8266 qua **HTTP Request**.

2.5. Hoạt động hệ thống

App gửi chuỗi → ESP8266 nhận và xử lý → LED Matrix hiển thị chạy chữ.

Chương II. THIẾT KẾ HỆ THỐNG

1. Kiến trúc tổng thể hệ thống

Hệ thống gồm 3 thành phần chính:

- **Ứng dụng Flutter trên thiết bị di động:** giao diện người dùng để nhập nội dung hiển thị.
- **ESP8266 NodeMCU:** thiết bị điều khiển trung tâm, có nhiệm vụ kết nối WiFi, xử lý dữ liệu từ app và điều khiển LED.
- **LED Matrix 32x8 sử dụng chip MAX7219:** hiển thị dòng chữ dạng cuộn.

Ba thành phần này kết nối và tương tác với nhau như sau:

- App Flutter gửi chuỗi văn bản đến địa chỉ IP của ESP8266 qua HTTP GET/POST request.
- ESP8266 đóng vai trò là Web Server, nhận dữ liệu, xử lý và hiển thị lên LED Matrix.
- LED Matrix giao tiếp với ESP8266 qua chuẩn SPI, hiển thị văn bản động (chạy từ phải sang trái).

2. Mô tả chi tiết luồng xử lý

- Người dùng nhập chuỗi văn bản vào app di động Flutter.
- App gửi chuỗi này đến ESP8266 thông qua HTTP
 - **[VD: `http://192.168.1.100/message?text=HELLO`]**
- ESP8266 nhận dữ liệu qua Web Server, xử lý và truyền tới LED Matrix.
- LED Matrix hiển thị nội dung chạy ngang từng ký tự.

3. Vai trò các thành phần

Thành phần	Chức năng chính
Flutter App	Giao diện nhập chữ, gửi dữ liệu qua HTTP
ESP8266 NodeMCU	Kết nối WiFi, nhận dữ liệu, điều khiển LED Matrix thông qua giao tiếp SPI
LED Matrix MAX7219	Hiển thị văn bản chạy chữ

Chương III. LẬP TRÌNH PHẦN CỨNG

1. Cấu hình Wifi và thiết lập Web Server trên ESP8266

1.1. Kết nối WiFi

- ESP8266 được cấu hình để kết nối vào mạng Wi-Fi với tên và mật khẩu Wi-Fi nội bộ do người dùng tự cấu hình.

```
const char* ssid = "Error 404"; // Tên WiFi mà ESP8266 sẽ kết nối
const char* password = "01983468"; // Mật khẩu WiFi
WiFiServer server(80); // Khởi tạo server với cổng 80

void setup() {
    Serial.begin(115200); // Khởi động giao tiếp Serial
    // Kết nối tới WiFi
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        // Hiển thị trạng thái kết nối
        delay(1000); // Chờ 1 giây trước khi kiểm tra lại
    }
    server.begin(); // Bắt đầu server
    Serial.println("WiFi connected"); // Thông báo khi kết nối thành công
    Serial.println(WiFi.localIP()); // In địa chỉ IP ra Serial Monitor
}
```

→ ESP8266 sẽ kết nối WiFi, mở Web server tại cổng 80.

Hình 2. Tên và mật khẩu Wifi tự cấu hình

- Sau khi kết nối thành công, địa chỉ IP nội bộ của ESP8266 được hiển thị ra Serial Monitor để biết nơi truy cập.

1.2. Thiết lập Web Server

- ESP8266 sử dụng *WiFiServer* để khởi tạo Web Server tại cổng 80:

```
WiFiServer server(80);

void setup() {
    // Code connect WiFi...
    server.begin();
}

void loop() {
    WiFiClient client = server.available();
    // Xử lý yêu cầu từ client
}
```

- Web Server này sẽ lắng nghe các kết nối từ ứng dụng gửi tới. Khi có request từ client, ESP8266 sẽ xử lý nội dung yêu cầu và thực hiện hành động tương ứng.

2. Điều khiển LED Matrix

2.1. Sử dụng thư viện

Dự án sử dụng thư viện LEDMatrixDriver để điều khiển LED Matrix. Đây là thư viện hỗ trợ giao tiếp với các module LED sử dụng chip điều khiển MAX7219 thông qua giao tiếp SPI.

```
#include <LEDMatrixDriver.hpp> // Thư viện điều khiển LED Matrix
#include <ESP8266WiFi.h> // Thư viện cho WiFi ESP8266
#include <WiFiClient.h> // Thư viện để xử lý client cho server
```

2.2. Khởi tạo LED Matrix

- **Số module:** `LEDMATRIX_SEGMENTS = 4` tương ứng với 4 module LED Matrix 8x8.
- **Chiều rộng:** `LEDMATRIX_WIDTH = 31`, tức 4 module gộp lại có chiều ngang 32 điểm ảnh (do chỉ số từ 0-31).
- **Chiều cao:** `LEDMATRIX_HEIGHT = 7` cho mỗi hàng 8 điểm ảnh (thường sử dụng 7 hàng để hiển thị ký tự rõ hơn).
- **Chân CS (LOAD):** LED Matrix sử dụng chân D8 (GPIO15).
- Khởi tạo đối tượng điều khiển:

```
const uint8_t LEDMATRIX_CS_PIN = 15; // Chân CS cho LED Matrix
const int LEDMATRIX_WIDTH = 31; // Chiều rộng LED Matrix
const int LEDMATRIX_HEIGHT = 7; // Chiều cao LED Matrix
const int LEDMATRIX_SEGMENTS = 4; // Số segment của LED Matrix
LEDMatrixDriver lmd(LEDMATRIX_SEGMENTS, LEDMATRIX_CS_PIN); // Khởi
tạo driver cho LED Matrix
LEDMatrixDriver lmd(LEDMATRIX_SEGMENTS, LEDMATRIX_CS_PIN);
```

2.3. Bộ font hiển thị

Font bitmap 8x8 được định nghĩa thủ công bằng mảng ***byte font[[[8]***, đại diện cho từng ký tự cần hiển thị (ví dụ: chữ cái, số, ký tự đặc biệt...).

```
// Block font 8x8
byte font[[[8]= {
    { 0,0,0,0,0,0,0,0 }, //space
    { 0x18, 0x3C, 0x3C, 0x18, 0x18, 0x00, 0x18, 0x00}, // U+0021 (!)
    // ... (các ký tự khác)
    { 0x00, 0x00, 0x3F, 0x19, 0x0C, 0x26, 0x3F, 0x00}, // U+007A (z)
};
```

3. Nhận dữ liệu từ app và hiển thị lên ma trận LED

3.1. Mã xử lý HTTP Request

Hàm *loop()* liên tục kiểm tra xem có client nào kết nối đến server không. Nếu có, nó sẽ đọc yêu cầu từ client, phân tích và trích xuất các tham số (như text, speed, brightness) và phản hồi lại một trang HTML cho phép người dùng điều khiển LED Matrix.

```

void loop() {
    // Xử lý yêu cầu từ client
    WiFiClient client = server.available(); // Kiểm tra xem có client nào kết nối không
    if (client) {
        String req = client.readStringUntil('\r'); // Đọc yêu cầu
        client.flush();

        // Parse the request
        if (req.indexOf('?') > 0) {
            String params = req.substring(req.indexOf('?') + 1); // Lấy các tham số trong
            yêu cầu
            // Xử lý các tham số như text, speed, brightness, v.v.
            // Ví dụ:
            if (params.indexOf("text=") >= 0) {
                String newText = params.substring(params.indexOf("text=") + 5); // Lấy giá
                trị của text
                // Xử lý văn bản mới
            }
            // ... (các tham số khác)
        }

        // Gửi lại phản hồi
        String s = "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n";
        s += "<!DOCTYPE html><html lang='en'><head><meta name='viewport'
        content='width=device-width, initial-scale=1'>";
        s += "<title>LED Matrix Control</title></head><body>";
        s += "<h1>LED Matrix Control</h1>";
        // Thêm form để cập nhật thông tin
        s += "</body></html>";
        client.print(s); // Gửi phản hồi lại cho client
    }
}

```

3.2. Hiện thị văn bản động (scroll text)

Hàm *displayText()* quản lý việc hiển thị văn bản trên LED Matrix. Nó kiểm tra trạng thái chạy, điều chỉnh thời gian và vẽ văn bản lên màn hình. Nếu văn bản đã cuộn hết, nó sẽ tăng số lần lặp lại và có thể dừng lại hoặc tiếp tục.

```
void displayText(char *theText) {  
    if (currentState != RUNNING) {  
        lmd.display();  
        return; // Không hiển thị nếu không ở trạng thái RUNNING  
    }  
  
    if (myTime + animationSpeed < millis()) {  
        myTime = millis(); // Cập nhật thời gian hiện tại  
        if (showText) {  
            drawString(theText, len, x, 0); // Vẽ chuỗi văn bản lên LED Matrix  
            lmd.display(); // Cập nhật LED Matrix  
        } else {  
            lmd.clear(); // Xóa màn hình nếu không hiển thị  
            lmd.display();  
        }  
        if (--x < len * -8) {  
            currentLoop++; // Tăng số lần lặp  
            // Kiểm tra số lần lặp lại  
        }  
    }  
}
```

4. Xử lý hiệu ứng

Hai hàm *drawString()* và *drawSprite()* chịu trách nhiệm vẽ các ký tự lên LED Matrix. Hàm *drawString()* sẽ lặp qua từng ký tự và gọi *drawSprite()* để vẽ nó, trong khi *drawSprite()* kiểm soát việc hiển thị từng pixel dựa trên dữ liệu font.

```
// Hàm để vẽ chuỗi văn bản lên LED Matrix
void drawString(char* text, int len, int x, int y) {
    for (int idx = 0; idx < len; idx++) {
        int c = text[idx] - 32; // Chuyển đổi ký tự thành chỉ mục font
        if (x + idx * 8 > LEDMATRIX_WIDTH)
            return; // Nếu vượt quá chiều rộng thì dừng
        if (8 + x + idx * 8 > 0)
            drawSprite(font[c], x + idx * 8, y, 8, 8); // Vẽ ký tự lên LED Matrix
    }
}

// Hàm để vẽ sprite lên LED Matrix
void drawSprite(byte* sprite, int x, int y, int width, int height) {
    byte mask = B10000000; // Khởi tạo mask
    for (int iy = 0; iy < height; iy++) {
        for (int ix = 0; ix < width; ix++) {
            lmd.setPixel(x + (width - ix), y + iy, (bool)(sprite[iy] & mask)); // Thiết lập pixel
            mask = mask >> 1; // Dịch trái mask
        }
        mask = B10000000; // Reset mask cho dòng tiếp theo
    }
}
```

5. Hiệu ứng hiển thị chữ trên LED matrix

5.1. Xử lý văn bản hiển thị

- Lấy chuỗi văn bản từ URL.
- Giải mã URL (nếu chứa ký tự %20, v.v...).
- Cập nhật chuỗi văn bản hiển thị.
- Khởi tạo lại các thông số để chạy lại từ đầu.

```

if (params.indexOf("text=") >= 0) {
    String newText = params.substring(params.indexOf("text=") + 5);
    if (newText.indexOf('&') > 0) {
        newText = newText.substring(0, newText.indexOf('&'));
    }
    newText = urlDecode(newText);
    strcpy(text, newText.c_str());
    len = strlen(text);
    x = LEDMATRIX_WIDTH;
    lmd.clear();
    displayOn = true;
    currentState = RUNNING;
    currentLoop = 0;
}

```

5.2. Điều chỉnh tốc độ chạy chữ

- Lấy giá trị tốc độ từ URL.
- Chuyển thành số nguyên.
- Đảm bảo tốc độ không thấp hơn giới hạn tối thiểu (10 ms/frame chẳng hạn).

```

if (params.indexOf("speed=") >= 0) {
    String speedStr = params.substring(params.indexOf("speed=") + 6);
    if (speedStr.indexOf('&') > 0) {
        speedStr = speedStr.substring(0, speedStr.indexOf('&'));
    }
    animationSpeed = speedStr.toInt();
    if (animationSpeed < 10) animationSpeed = 10;
}

```

5.3. Điều chỉnh độ sáng

- Đọc giá trị độ sáng từ URL.
- Giới hạn trong khoảng 0–15 (theo độ sáng của LED matrix driver).
- Gửi giá trị đến LED matrix để điều chỉnh sáng.

```
if (params.indexOf("brightness=") >= 0) {
    String brightnessStr = params.substring(params.indexOf("brightness=") + 11);
    if (brightnessStr.indexOf('&') >= 0) {
        brightnessStr = brightnessStr.substring(0, brightnessStr.indexOf('&'));
    }
    brightness = brightnessStr.toInt();
    if (brightness < 0) brightness = 0;
    if (brightness > 15) brightness = 15;
    lmd.setIntensity(brightness);
}
```

5.4. Thiết lập số lần lặp chạy chữ

- Nhận số lần lặp từ URL.
- Đảm bảo ít nhất lặp 1 lần.
- Reset bộ đếm để bắt đầu lại từ vòng lặp đầu tiên.

```
if (params.indexOf("loops=") >= 0) {
    String loopsStr = params.substring(params.indexOf("loops=") + 6);
    if (loopsStr.indexOf('&') > 0) {
        loopsStr = loopsStr.substring(0, loopsStr.indexOf('&'));
    }
    numLoops = loopsStr.toInt();
    if (numLoops < 1) numLoops = 1;
    currentLoop = 0;
}
```

5.5. Bật/tắt nhấp nháy

- Đọc giá trị blink.
- Nếu bằng "1" → bật nhấp nháy.
- Ngược lại → tắt nhấp nháy.

```

if (params.indexOf("blink=") >= 0) {
    String blinkStr = params.substring(params.indexOf("blink=") + 6);
    if (blinkStr.indexOf('&') >= 0) {
        blinkStr = blinkStr.substring(0, blinkStr.indexOf('&'));
    }
    if (blinkStr == "1") {
        enableBlink = true;
    } else {
        enableBlink = false;
    }
}

```

5.6. Lệnh dừng hiển thị

Nếu nhận $stop=1 \rightarrow$ chuyển trạng thái hiển thị sang STOPPED.

```

if (params.indexOf("stop=") >= 0) {
    String stopStr = params.substring(params.indexOf("stop=") + 5);
    if (stopStr.indexOf('&') >= 0) {
        stopStr = stopStr.substring(0, stopStr.indexOf('&'));
    }
    if (stopStr == "1") {
        currentState = STOPPED;
    }
}

```

Chương IV. LẬP TRÌNH ỨNG DỤNG FLUTTER

1. Giới thiệu

Flutter là framework phát triển ứng dụng giao diện người dùng (UI toolkit) do Google phát triển, sử dụng ngôn ngữ Dart. Flutter cho phép xây dựng ứng dụng mobile (Android/iOS) từ một codebase duy nhất với hiệu năng cao.

Trong đồ án này, Flutter được sử dụng để phát triển một ứng dụng mobile điều khiển ma trận LED 32x8 qua Wi-Fi (ESP8266). Ứng dụng cho phép người dùng:

- Nhập nội dung cần hiển thị.
- Chính tốc độ cuộn, độ sáng, số lần lặp.
- Gửi dữ liệu đến thiết bị ESP8266.
- Dừng và làm nhấp nháy văn bản.

2. Mục tiêu ứng dụng Flutter

Ứng dụng Flutter được xây dựng nhằm mục tiêu:

- Cung cấp giao diện thân thiện cho người dùng thao tác.
- Gửi dữ liệu điều khiển LED Matrix thông qua giao thức HTTP.
- Cho phép điều chỉnh linh hoạt các thông số hiển thị:
 - + Văn bản
 - + Tốc độ cuộn (ms)
 - + Độ sáng (0–15)
 - + Số lần lặp lại
 - + Chế độ nhấp nháy
- Thêm nút dừng nội dung (Stop).

3. Giao diện người dùng (UI)

3.1. Cấu trúc giao diện bao gồm các thành phần:

Thành phần	Mô tả
TextField	Nhập văn bản hiển thị
TextField	Nhập tốc độ chạy chữ (đơn vị ms)
TextField	Nhập độ sáng từ 0 đến 15
TextField	Nhập số lần lặp (loops)
SwitchListTile	Bật/tắt chế độ nhấp nháy văn bản

Thành phần	Mô tả
ElevatedButton	Gửi dữ liệu hiển thị đến ESP8266
ElevatedButton	Nút Stop để ngừng hiển thị nội dung trên LED

3.2. Thiết kế giao diện trực quan và dễ thao tác:

Nhập nội dung (Text)	← TextField
Tốc độ chạy chữ (ms)	← TextField
Độ sáng (0-15)	← TextField
Số lần lặp	← TextField
[Switch] Nhấp nháy văn bản	← Switch
[Gửi dữ liệu đến LED]	← Gửi nội dung
[Stop]	← Nút dừng hiển thị

4. Luồng xử lý dữ liệu

- Người dùng nhập các thông số như: văn bản, tốc độ, độ sáng, số lần lặp, bật/tắt nhấp nháy.
- Nhấn nút "Send to LED Matrix".
- Ứng dụng sử dụng HTTP (method GET) để gửi yêu cầu đến địa chỉ IP của ESP8266.
- ESP8266 phân tích tham số nhận được, và hiển thị nội dung lên LED Matrix theo cấu hình.

5. Giải thích mã nguồn chính

5.1. Các trường dữ liệu điều khiển

```
final TextEditingController _textController;    // Văn bản
final TextEditingController _speedController;  // Tốc độ chạy chữ
final TextEditingController _brightnessController; // Độ sáng LED
final TextEditingController _loopsController;  // Số lần lặp
bool _blink = false;                          // Trạng thái nhấp nháy
```

5.2. Hàm gửi dữ liệu tới ESP8266

```
Future<void> _sendData({bool stop = false, bool pause = false, bool resume = false})
```

- Sử dụng *http.get()* để gửi dữ liệu thông qua HTTP query parameters.

5.3. Tạo URI từ các tham số

```
final uri = Uri.http('$_ipAddress', '/', queryParams);
```

6. Cài đặt thư viện và cấu hình Flutter

- Cài đặt thư viện HTTP:

```
flutter pub add http
```

- Các import sử dụng:

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
```

7. Kiểm thử và triển khai

7.1. Yêu cầu

- Thiết bị Android và ESP8266 phải nằm trong cùng một mạng Wi-Fi.
- Xác định đúng địa chỉ IP của ESP8266 (ví dụ: 192.168.1.103).

7.2. Thử nghiệm

- Gửi các chuỗi văn bản có độ dài khác nhau.
- Kiểm tra phản hồi từ ESP quaSnackBar.
- Thử nghiệm với chế độ blink, dừng và lặp.

Chương V. KẾT LUẬN

Trong đồ án “Hiển thị chạy chữ trên LED Matrix 32x8 sử dụng ESP8266 qua mobile app”, nhóm đã hoàn thành toàn bộ quá trình thiết kế, lập trình và triển khai hệ thống điều khiển LED Matrix thông qua ứng dụng Flutter. Cụ thể, nhóm đã:

- Tìm hiểu và ứng dụng vi điều khiển ESP8266 NodeMCU trong việc giao tiếp với LED Matrix MAX7219.
- Thiết lập Web Server trên ESP8266 để tiếp nhận dữ liệu từ điện thoại thông qua giao thức HTTP.
- Lập trình ứng dụng Flutter mobile cho phép người dùng nhập nội dung văn bản, điều chỉnh tốc độ, độ sáng, số lần lặp và bật/tắt chế độ nhấp nháy, sau đó gửi dữ liệu đến ESP8266.
- Thực hiện kiểm thử hệ thống trong mạng LAN và đảm bảo hoạt động ổn định.

Kinh nghiệm và bài học rút ra

- Nắm vững kiến thức về giao tiếp giữa thiết bị IoT và ứng dụng di động thông qua mạng Wi-Fi.
- Thành thạo hơn trong việc lập trình ESP8266 bằng Arduino IDE và xây dựng app với Flutter SDK.
- Rèn luyện kỹ năng xử lý lỗi thực tế trong quá trình kết nối mạng, xử lý HTTP và tương tác thiết bị phần cứng.

Hạn chế và khó khăn

- Hệ thống hiện chỉ hoạt động trong mạng LAN nội bộ, chưa hỗ trợ điều khiển qua Internet.
- Dữ liệu truyền qua giao thức HTTP (phương thức GET) nên chưa bảo mật cao.
- Ứng dụng chưa có giao diện hiển thị trạng thái phản hồi từ ESP (ví dụ như LED đang chạy hay đã dừng).

Hướng phát triển trong tương lai

- Nâng cấp giao diện người dùng bằng cách bổ sung trạng thái hiển thị, lịch sử nội dung đã gửi.
- Chuyển sang giao thức POST hoặc tích hợp WebSocket để tăng tính bảo mật và hiệu năng truyền nhận dữ liệu.
- Hỗ trợ điều khiển từ xa qua Internet, tích hợp nền tảng như Firebase hoặc MQTT.
- Mở rộng hệ thống để hiển thị hình ảnh đơn giản hoặc biểu tượng thay vì chỉ văn bản.